📦 Stocker AI

Project flowcharts

How data moves through Stocker AI, the inventory and re-ordering assistant for small shops: from sales files and photos, through forecasts and advice, to orders, alerts and answers.

1. The whole system
2. Forecasting: choosing a method for each product
3. Plan & order: from needs to suppliers
4. Photo import: receipts, invoices and lists
5. Ask Stocker AI: typed and spoken questions
6. Day to day: stock, alerts and looking back

How to read the charts

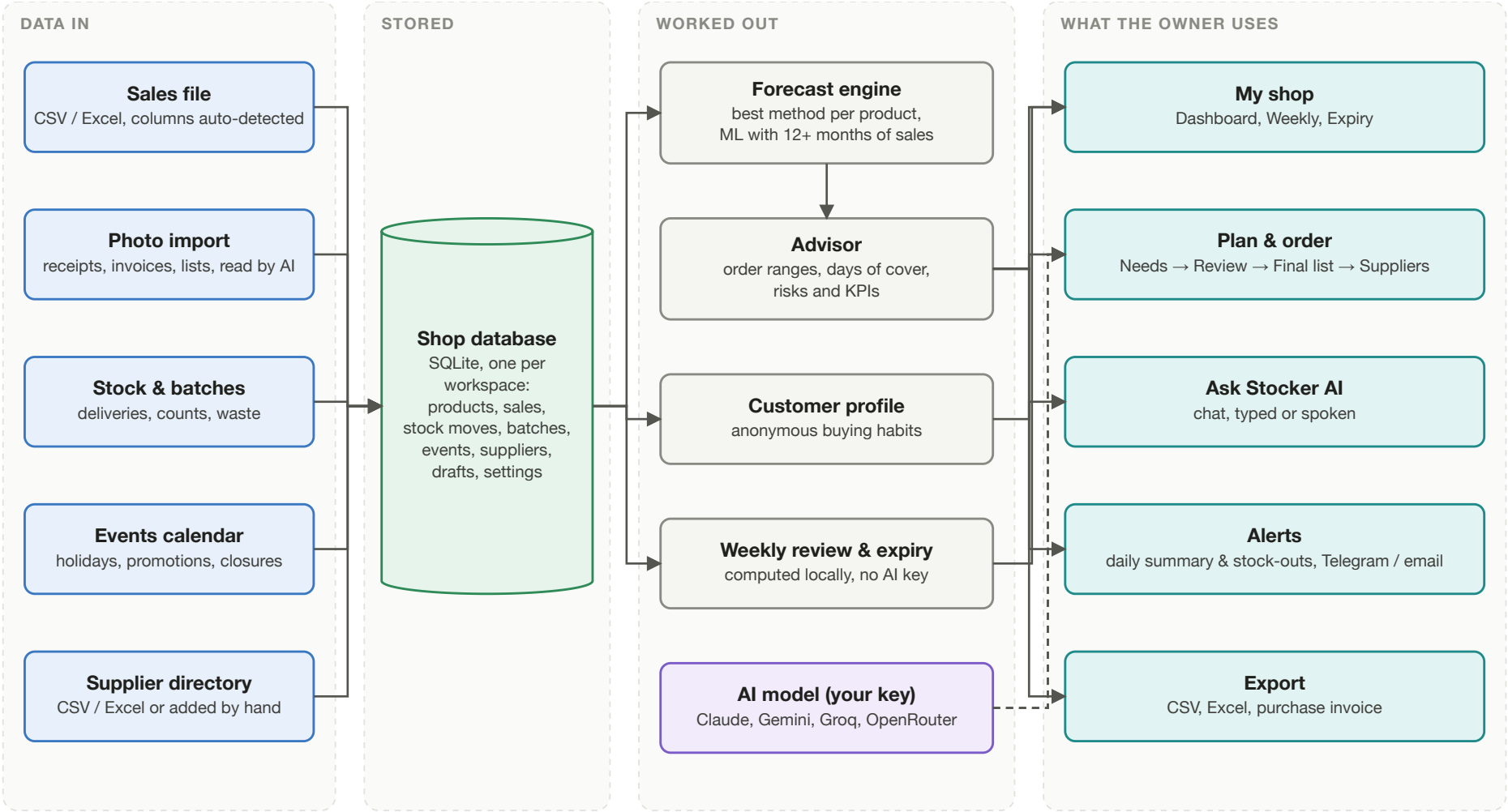
Data in	**Program**	**AI model**	**Owner / check**	**Saved**	**Owner sees**	**Refused**
files, photos, counts	worked out by code	optional key	a decision or yes-no	in the database	shown or sent	must be fixed

Built with Python, Streamlit, SQLite, pandas and scikit-learn. The AI model is optional: Claude, Gemini, Groq or OpenRouter, with the owner's own key.

Arrows show the order of steps. Dashed arrows are optional or happen later. Version: StockerAI-v9 with the forecast-method, photo-import, voice and connected plan & order updates · November 2026.

1 The whole system

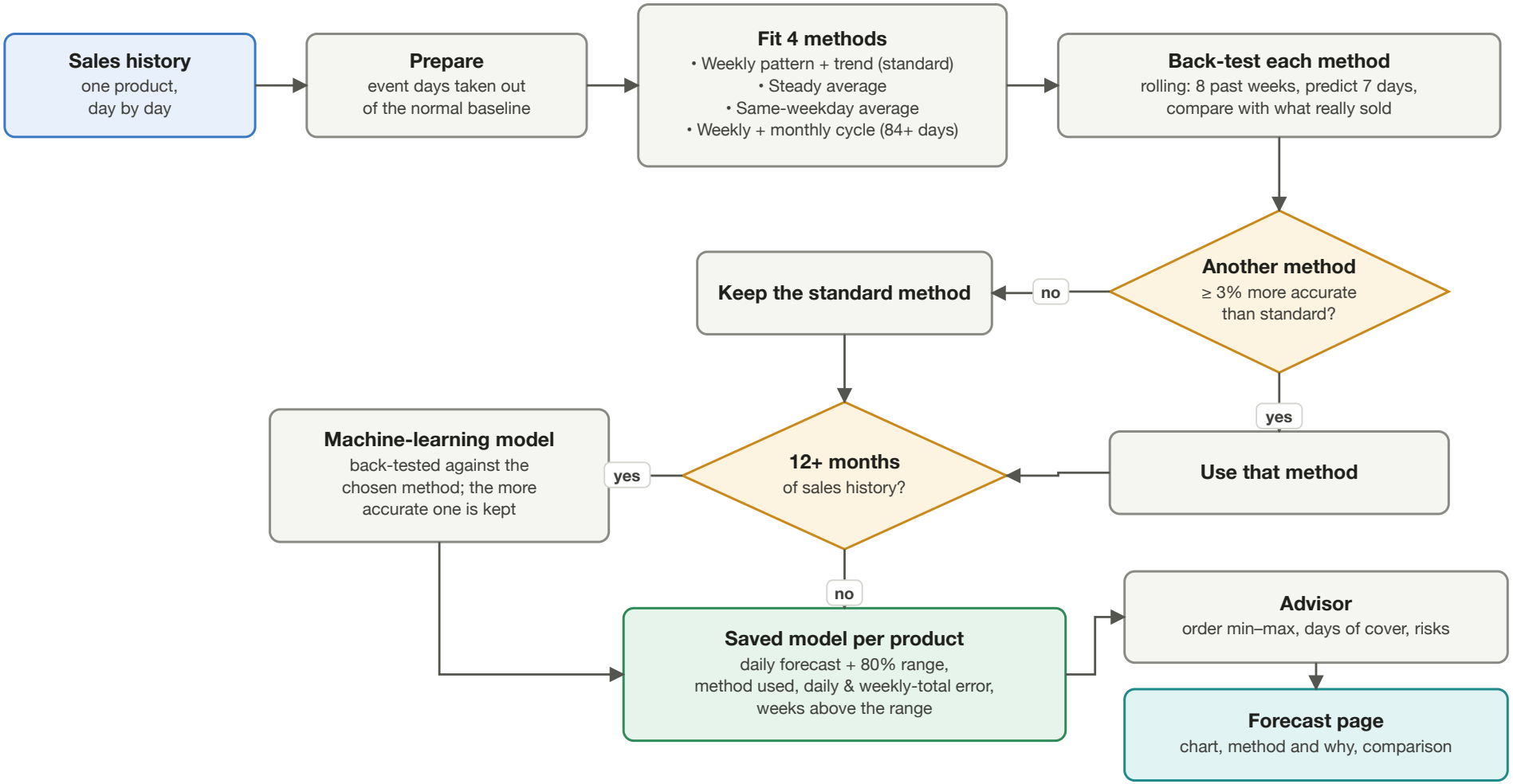
Shop data comes in on the left and is kept in one database per shop. The program works out forecasts, advice and risks; the owner sees and acts on them on the right. The AI model (dashed lines) is optional: it reads photos, drafts plans and answers questions, but every number it uses comes from the database.



- Without an AI key, everything except photo reading, AI plans and chat still works.
- Workspaces are separate shops: each has its own database file under `data/workspaces/`.

2 Forecasting: choosing a method for each product

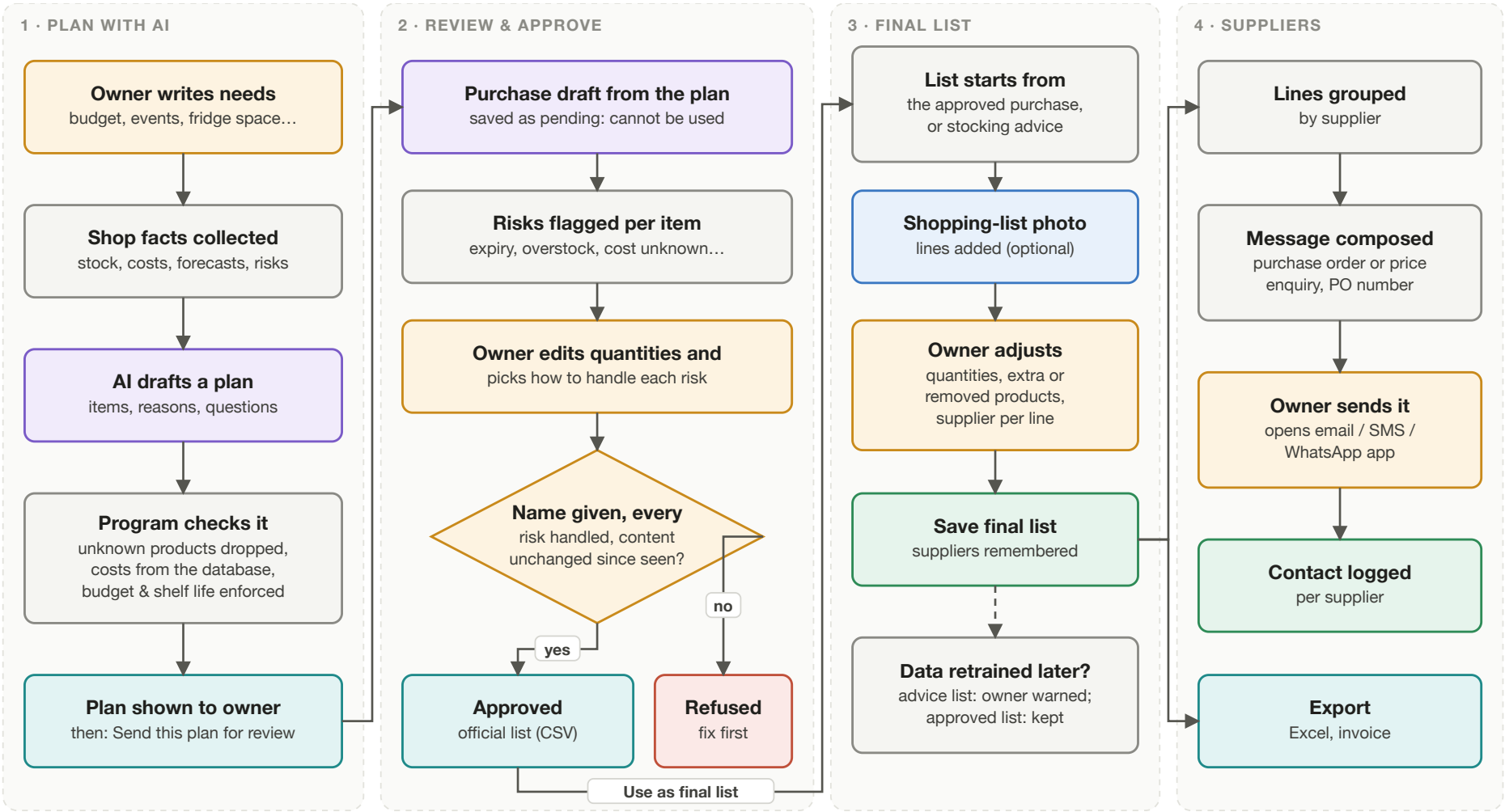
Products sell differently, so every product gets the method that predicted its own past sales best. The standard method stays unless another one is clearly better.



- Runs when data is imported and when the owner presses **Retrain models**.
- The weekly-total error is the main accuracy measure, because re-orders are made for a week, not a day.
- A weak monthly pattern is shrunk toward none, so a shop without paydays is not given one.

3 Plan & order: from needs to suppliers

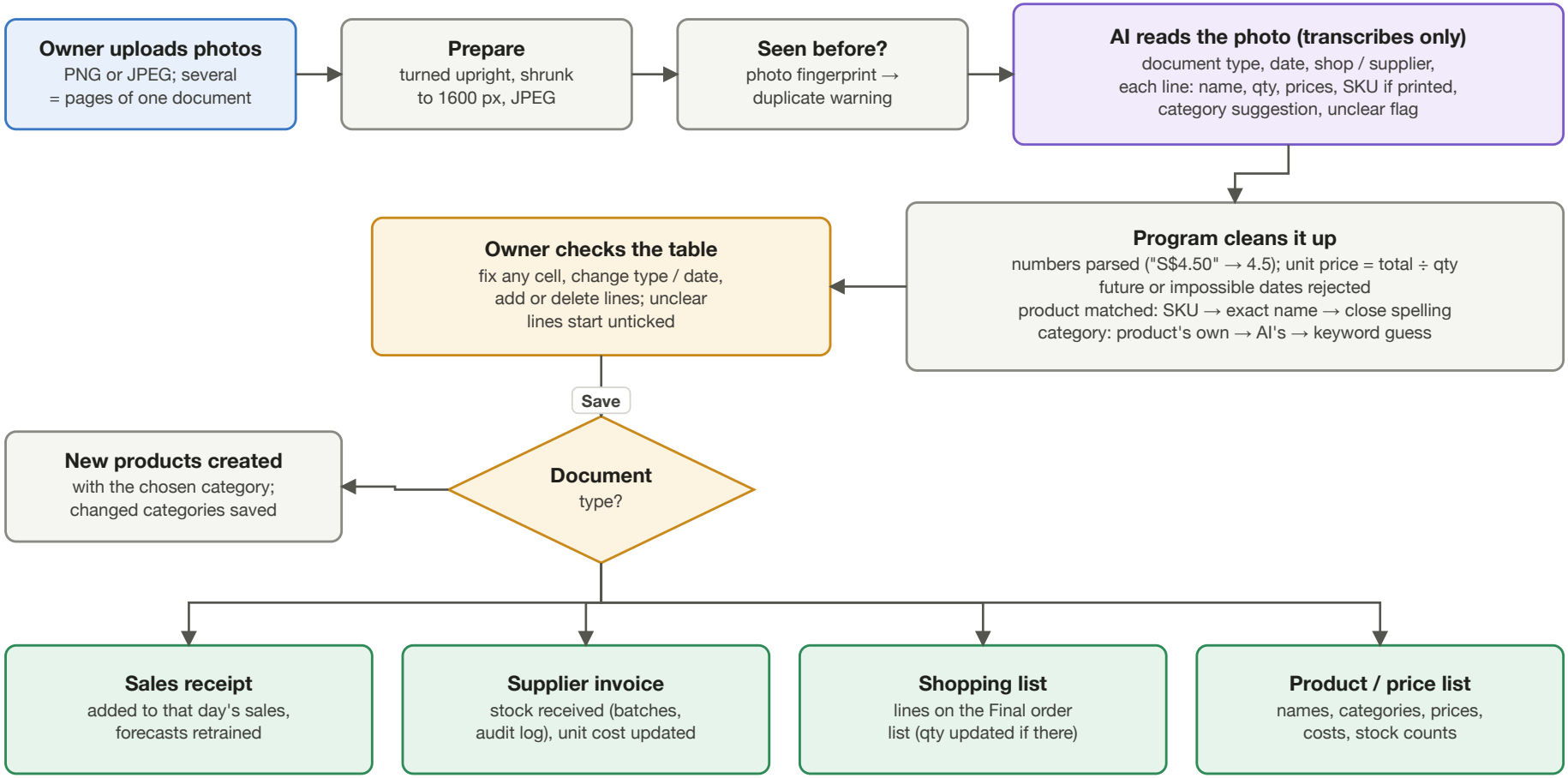
The four steps of the **Plan & order** menu. The AI only suggests; the program checks the numbers, and the owner makes every decision. Each step hands its result to the next with one button. Nothing is sent without the owner pressing a button.



- Step 1 → 2: **Send this plan for review** (`review.from_plan`). Step 2 → 3: **Use as my final order list**, also offered on the Order list page (`orders.from_approved`). Step 3 → 4: the saved list is grouped by supplier.
- Review rules live in `stocker/review.py` , so no screen can bypass them: editing an approved draft cancels the approval, and a new draft replaces the old one. Every step is logged.
- Approving a draft or saving a list never changes stock. Stock goes up only when goods are received (a delivery, or a supplier invoice photo).

4 Photo import: receipts, invoices and lists

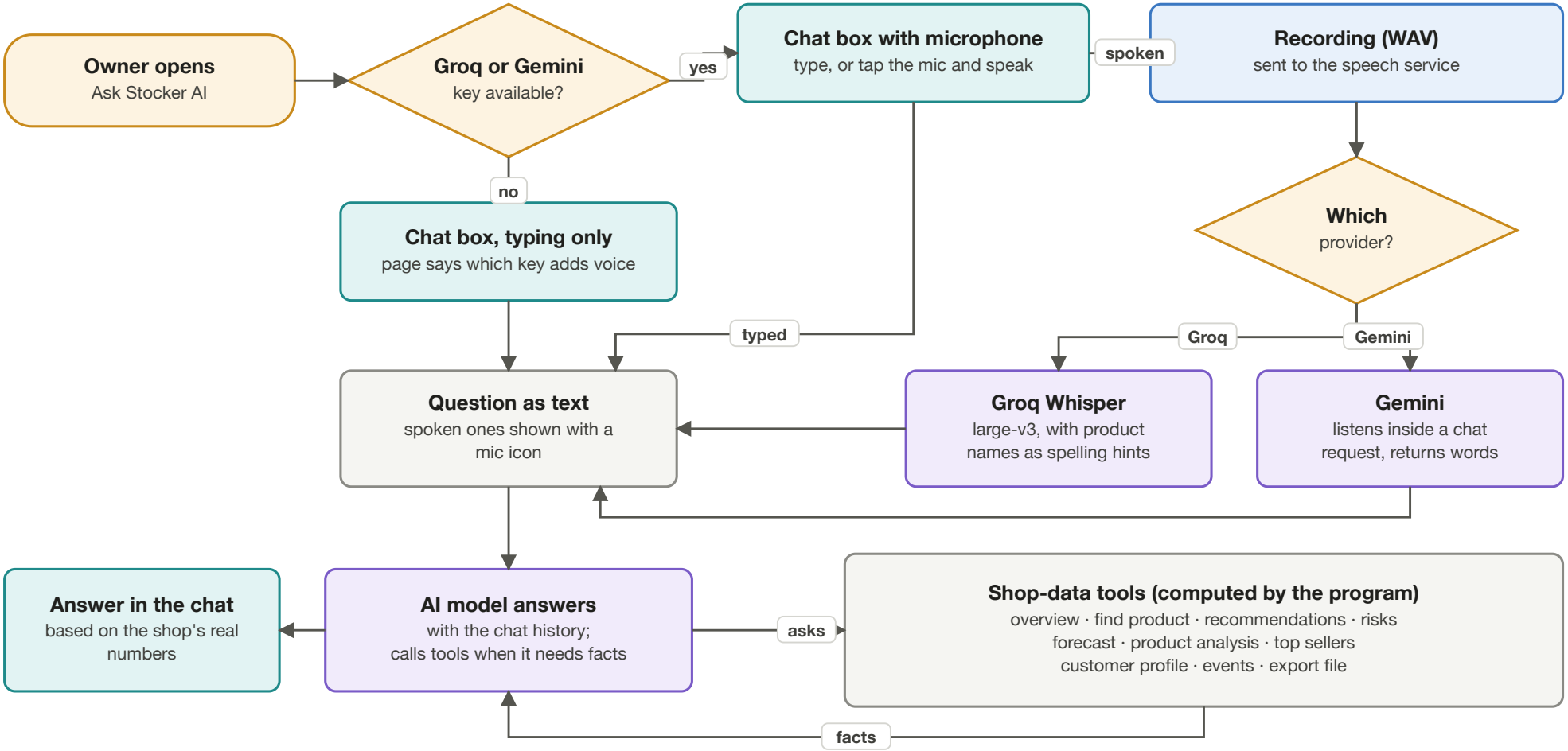
Import data → **Photo**. The AI only copies what is written; the program does the matching and the maths, and nothing is saved until the owner presses **Save to Stocker AI**.



- Lines without a quantity are refused. Photos are not stored; only their fingerprints, to warn about duplicates.
- Code: `stocker/scan.py` (read, tidy, match, save) and `views/data.py` (the page).

5 Ask Stocker AI: typed and spoken questions

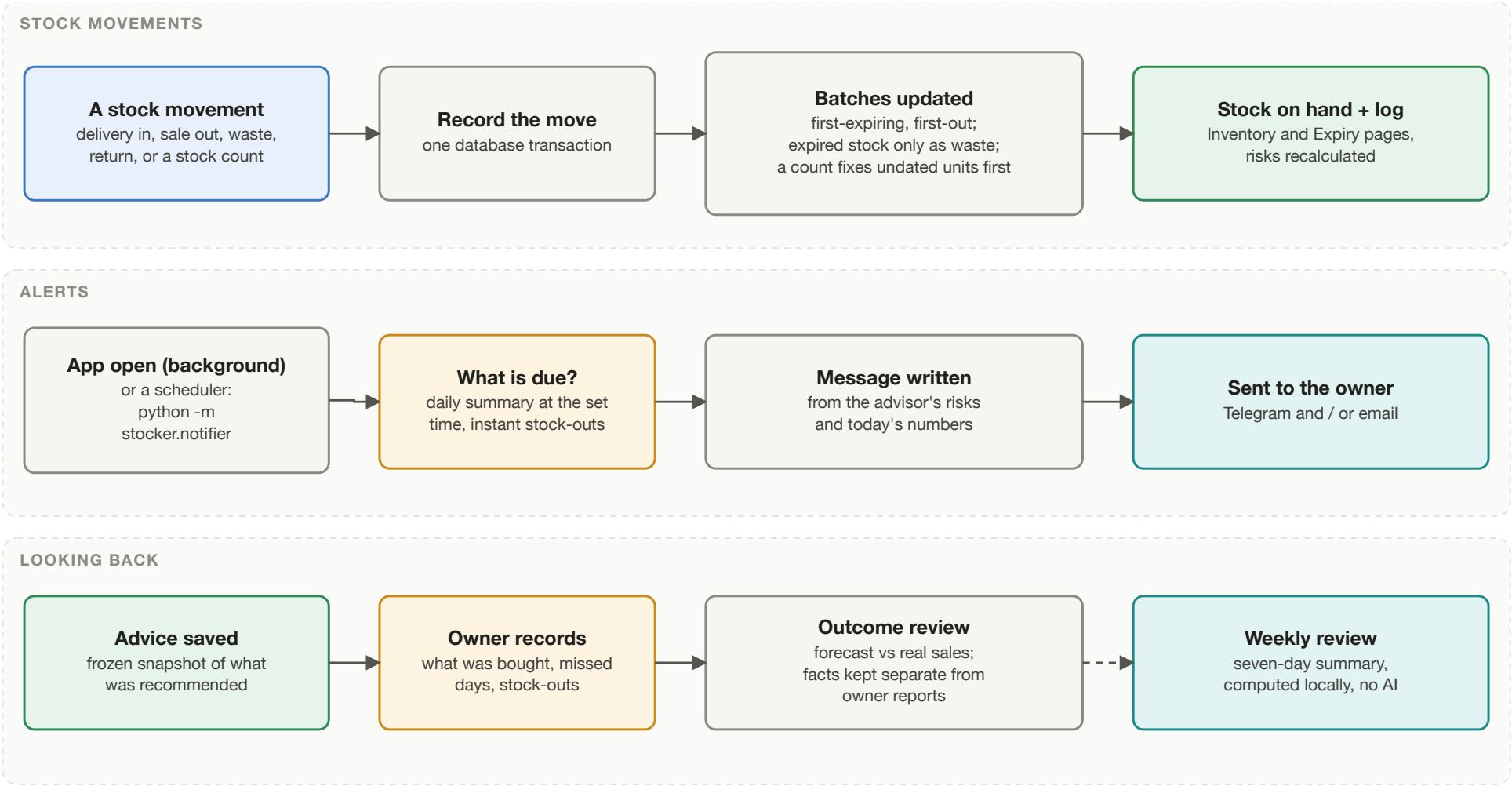
Questions can be typed or spoken. A recording is turned into text first, then answered like a typed question. The model looks up facts through tools instead of guessing numbers.



- Claude and OpenRouter keys answer questions but cannot hear audio. With a Claude key, voice works when a `GROQ_API_KEY` or `GEMINI_API_KEY` is also set on the server.
- Code: `views/assistant.py` (chat box and mic) and `stocker/agent.py` (`transcribe()`, `chat()`, tools).

6 Day to day: stock, alerts and looking back

Three things that run alongside planning: every stock change goes through one place, alerts reach the owner even with the app closed, and past advice is checked against what really happened.



- Each product is **Perishable** or **Non-perishable**: chilled or frozen, a shelf life of 30 days or less, or a fresh-food category counts as perishable. The owner can change it on the Inventory page.
- Importing a sales file adds sales history only; it never uses up current batches.
- Missing sales days are treated as gaps, not as zero sales, unless the owner confirms them.
- Code: `stocker/batches.py`, `db.record_move()`, `stocker/notifier.py`, `stocker/evidence.py`, `stocker/weekly.py`.